

Optimierung von Beschleunigungsrampen und Dämpfungsverfahren für Schrittmotoren

Von analoger Frequenzmodulation (SMIK) bis zur f - t -Kalkulation in JavaMotion

Autoren:

- Martin Habenicht, René Merz (Reinton Audio Lab, reinton.ch)

Projekt: MUFACE v6 Public Release

Abstract

*Die dynamische Ansteuerung von Schrittmotoren steht seit jeher im Spannungsfeld zwischen maximaler Dynamik, hoher Energieeffizienz und der Vermeidung von Resonanz-Instabilitäten. Während historische Pioniere wie Pierre Boillat Ende der 1980er Jahre mit Verfahren wie der SMIK-Methode (Stepping Motor Instability Killer) analoge Schaltungen nutzten, um Motorinstabilitäten direkt im Steuersignal zu dämpfen, verlagert das Framework **JavaMotion (MUFACE v6)** diese Aufgabe vollständig in die diskrete Mathematik. Dieses Paper dokumentiert die Evolution von den analogen NE555-Modulationsschaltungen über diskrete Phasenmodulatoren bis hin zur präzisen f - t -Rampenkalkulation in Java.*

1. Einleitung & Historischer Rückblick

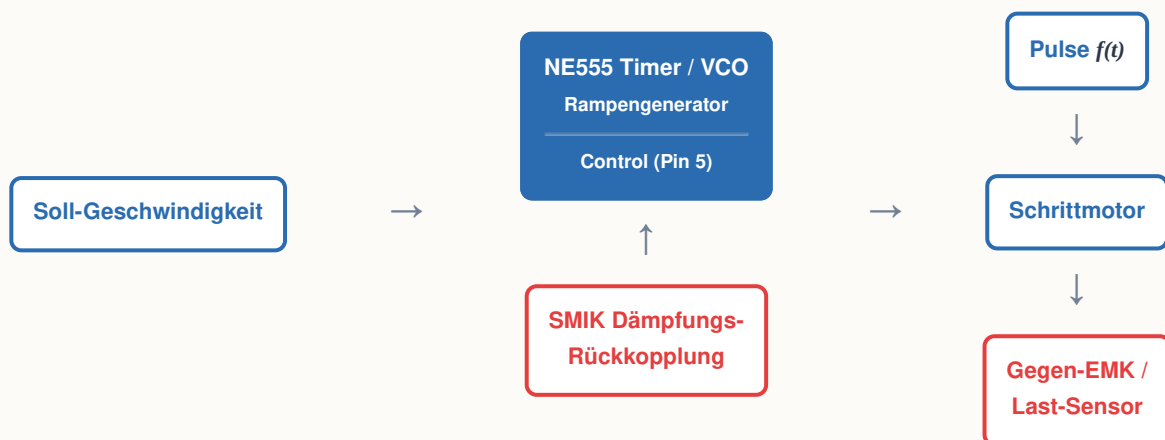
Schrittmotoren neigen aufgrund ihres physikalischen Aufbaus (Feder-Masse-System aus Rotor-Trägheitsmoment und magnetischem Haltemoment) in bestimmten Frequenzbereichen zu Resonanzen. Ohne Dämpfung führt dies zu Schrittverlusten, starker Lärmentwicklung oder dem vollständigen Ausrasten des Motors. Zudem werden viele Schrittmotoren aus Sicherheitsgründen weit unter ihren Möglichkeiten betrieben – meist im uneffizienten Start-Stopp-Bereich (Pull-In, <500 Hz) mit Wirkungsgraden von unter 10 %.

1.1 Die SMIK-Methode und das Boillat-Patent (1987)

Auf dem ETHZ Mechatronik Symposium 1987 stellte Pierre Boillat (damals Sodeco SAIA AG, heute [Johnson Electric International](#)) mit der SMIK-Methode (*Stepping Motor Instability Killer*) einen wegweisenden Ansatz vor, der eng mit seinem Patent (US 4,683,409) verknüpft ist.

Schrittmotoren neigen nicht nur zu linearen Eigenresonanzen, sondern vor allem zu **parametrischen Resonanzen** (parametrische Oszillationen des Rotors im Midband-Bereich). Diese entstehen durch die zyklische Änderung der magnetischen Steifigkeit während des Schrittbetriebs. Wenn die Ansteuerfrequenz in einem bestimmten Verhältnis zur mechanischen Eigenfrequenz steht, wird dem Rotor-System periodisch Energie zugeführt, was zu einer schnellen, unkontrollierten Instabilität führt.

Die von Pierre Boillat entwickelte SMIK-Methode setzt genau hier an: Anstatt die parametrische Oszillation rein mechanisch zu dämpfen, wird über den Control-Eingang (Pin 5 des Timer-Bausteins NE555) eine gezielte Frequenz- und Phasenmodulation eingespeist. Diese zerstört die Phasenstarrheit der parametrischen Anregung, sodass der Rotor keine Energie aus dem Feld für die Oszillation aufbauen kann. Die Kernidee war es, Instabilitäten aktiv im elektrischen Ansteuersignal zu bekämpfen:



Funktionsweise der analogen Schaltung:

- **Lineare Rampe via Stromquelle:** Ein NE555-Timer wird nicht über einfache Widerstände geladen, sondern über eine regelbare Stromquelle. Dies erzeugt eine exakt lineare Kondensator-Ladespannung $U(t)$ und damit saubere Anlauf- und Stopprampen.
- **Modulation über Pin 5 (Control Voltage):** Boillat nutzte den direkten Zugriff auf den internen Spannungsteiler des NE555 (Pin 5). Indem ein rückgekoppeltes Signal (z. B. aus der Gegen-EMK) auf Pin 5 eingespeist wurde, konnte die Umschalt-Schwelle dynamisch variiert werden.
- **Frequenz- und Phasenmodulation (FM/PM):** Traten im Motor Schwingungen auf, modulierte die SMIK-Schaltung die Schrittfrequenz im Mikrosekundenbereich gezielt gegen

die mechanische Eigenbewegung des Rotors. Die Resonanzenergie wurde elektrisch aus dem System gezogen.

2. Digitale Evolutionsstufen & Hardware-Nachrüstung

2.1 Zeitdiskrete Timer-Berechnung

Mit dem Aufkommen von Mikrocontrollern wurde die Frequenzgenerierung digitalisiert. Die Herausforderung verlagerte sich auf die zeitechte Berechnung von Schrittabständen:

$$\Delta t_n = t_n - t_{n-1}$$

Klassische Algorithmen (z. B. nach David Austin) nutzen Taylor-Approximationen, um die Wurzelfunktion für konstante Beschleunigung in Interrupt-Service-Routinen (ISR) schnell genug zu lösen.

2.2 Direct Digital Synthesis (DDS)

Um Timer-Jitter bei hohen Schrittraten zu vermeiden, nutzt man DDS. Ein Akkumulator summiert mit festem Takt ein Frequenzsteuerwort (FCW):

$$f_{\text{out}} = \frac{\text{FCW} \cdot f_{\text{clock}}}{2^N}$$

Damit lassen sich beliebig geformte f - t -Profile ohne mathematische Engpässe in Echtzeit ausgeben.

2.3 Nachrüstung bestehender Microcontroller via Phasenmodulator

In vielen bestehenden oder kostensensitiven Industrie-Architekturen ist weder ein NE555 IC noch ein direkter Frequenz-/Phasenmodulationseingang am Microcontroller verfügbar. Wie von Merz (2019) gezeigt (Manuskript-Draft 2018), lässt sich eine solche Steuerung extern „aufmotzen“: Ein eigenständiger **Pulsphasenmodulator (PM)** wird zwischen den Clock-Ausgang des Microcontrollers (CLK-IN) und den Sequenzer-Eingang des Motortreibers (CLK-PM) geschaltet. Das Rotoroszillationssignal wird an den Shunts des Drivers als Stromsignal $U = f(I_T)$ abgegriffen, gefiltert und steuert über einen Sägezahn-Komparator die Phasenlage der Schritimpulse dynamisch nach. Dadurch lässt sich die maximale mechanische Abgabeleistung $P_{\text{mech}} = M_d \cdot \omega$ um mehr als den **Faktor 3.2** steigern (z. B. von 1,75 W auf 5,6 W bei Tin-Can-Motoren), indem die kritischen Resonanzbänder sicher durchfahren und hohe Arbeitsfrequenzen (1000–3000 Hz) erreicht werden.

3. JavaMotion (MUFACE v6): Das moderne Framework

Die praktische Entwicklung und Validierung des JavaMotion-Frameworks erfolgte im Hardware-Lab über intensive Test- und Debugging-Zyklen direkt an den Motorprüfständen. Die algorithmische Kernumsetzung von JavaMotion stammt von Adrian Perrig, unter punktueller mathematischer Beratung durch Marc Fasnacht (EPFL) bei der Formulierung der iterativen Solver-Verfahren.

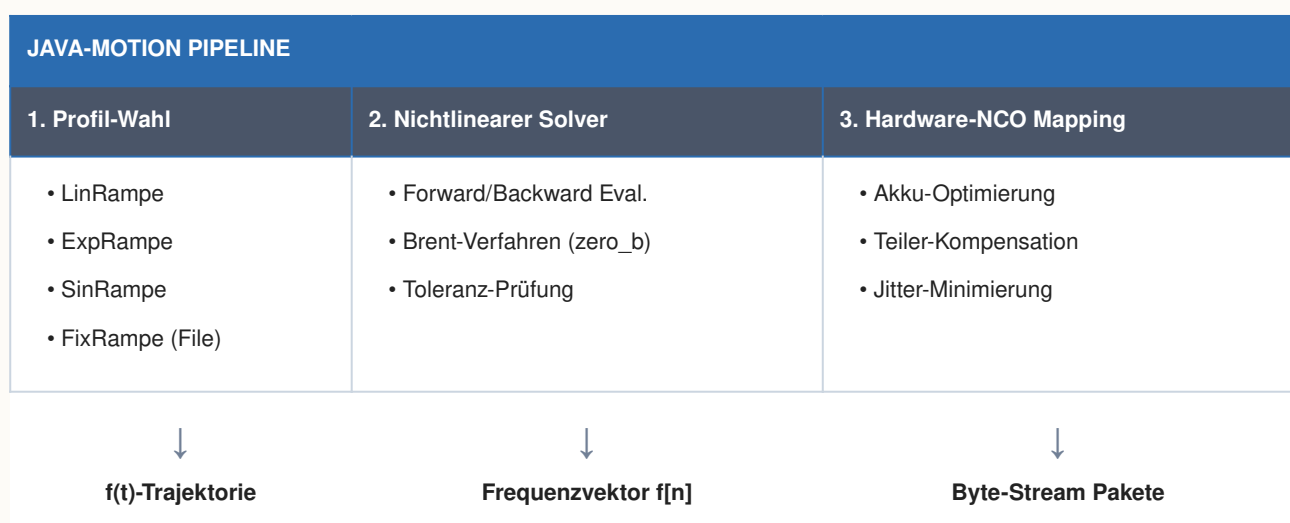
Das Framework JavaMotion bildet das mathematische Herzstück von MUFACE v6. Es überführt kontinuierliche physikalische Trajektorien in diskrete Steuersignale. Im Gegensatz zu vereinfachten Online-Algorithmen trennt JavaMotion die Profilgenerierung in eine zeit- und frequenzkonforme Solver-Stufe sowie eine Hardware-Synthese-Stufe mittels NCO (Numerically Controlled Oscillator).

3.1 Mathematische Modellierung & Profilgenerierung

Ein zentrales Problem bei der diskreten Schrittmotor-Ansteuerung liegt in der korrekten Interpretation des Schrittindexes i : Ein Schritt entspricht nicht einem konstanten Zeitintervall Δt , sondern einer diskreten Distanz (einem Winkelschritt). Die Ausgangsfunktion für die Schrittzeitpunkte lautet:

$$t_n = t_{n-1} + \Delta t_n = t_{n-1} + \frac{1}{f_n}$$

JavaMotion modelliert die Trajektorie daher über eine nichtlineare implizite Gleichungssystem-Lösung für gegebene Start- (f_0) und Endfrequenzen (f_{end}) bei fest vorgegebener Schrittzahl N .



3.1.1 Die unterstützten Profil-Topologien

1. **Sinus-Rampe (SinRampe):** Genutzt für extrem sanftes Ruck-Verhalten an den Bewegungsgrenzen (Viertel-Sinus-Harmonische):

$$f(i) = f_{\text{start}} + (f_{\text{stop}} - f_{\text{start}}) \cdot \sin\left(\frac{i}{N-1} \cdot \frac{\pi}{2}\right) \quad \text{für } i \in [0, N-1]$$

2. **Lineare Beschleunigung (LinRampe):** Strebt eine konstante mechanische Beschleunigung a an. Da $f(t)$ linear wachsen soll, müssen die Zeitabstände Δt_n quadratisch gelöst werden:

$$t_n = \text{quad}(c, -c \cdot t_{n-1} + f_{\text{start}}, -f_{\text{start}} \cdot t_{n-1} - 1.0)$$

wobei über eine implizite Rekursion die Steigung c bestimmt wird. Die Nullstellenbestimmung der Zeitabstände via `quad( . . . )` löst die diskrete Eigenzeit des Schrittmotors exakt auf.

3. **Exponentielle Rampe (ExpRampe):** Passt das Beschleunigungsverhalten an das exponentiell abfallende Drehmoment an. Für positive Rampen wird eine physikalisch stabile Rückwärts-Iteration eingesetzt:

$$f_i = f_{\infty} - (f_{\infty} - f_{i+1}) \cdot e^{-c / f_{i+1}}$$

mit der asymptotischen Grenzfrequenz $f_{\infty} = f_{\text{end}} + \text{sign} \cdot (f_{\text{end}} - f_{\text{start}}) \cdot \varepsilon_{\text{exp}}$. Dass die Berechnung rückwärts vom Zielwert abgewickelt wird, verhindert numerische Divergenzen.

4. **Tabellarische Trajektorie (FixRampe):** Ermöglicht das Einlesen extern berechneter oder aus Simulationen stammender Verläufe.

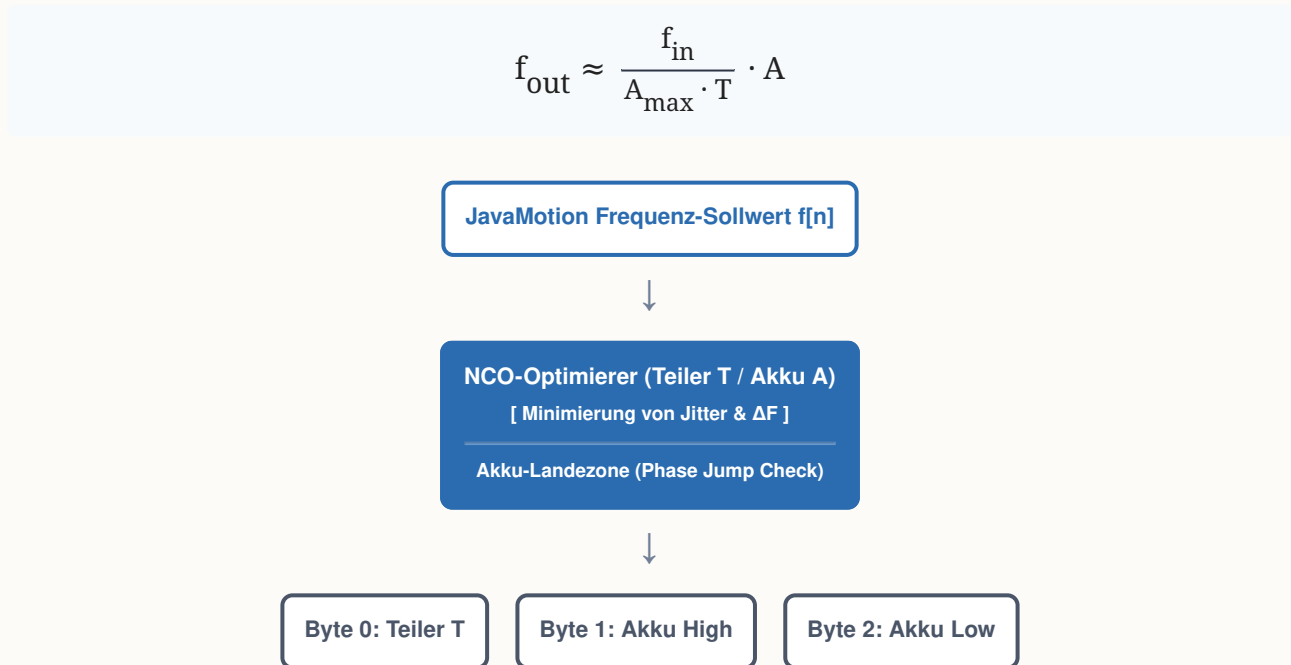
3.1.2 Numerische Optimierung via Brent-Verfahren

Da die Parameter (wie die Steigung c) für eine exakt vorgegebene Schrittzahl N und Ziel-Endfrequenz f_{end} nicht in geschlossener Form auflösbar sind, nutzt JavaMotion das Root-Finding-Verfahren nach Richard P. Brent (`zero_brent`).

- **Fehlerkriterium:** Der Solver sucht die Nullstelle der Funktion $g(c) = f_{\text{berechnet}}^{(N)} - f_{\text{target}}$
- **Kombinations-Strategie:** Das Verfahren vereint die Zuverlässigkeit der Bisection (Intervallhalbierung) mit der schnellen Konvergenz der Sekanten-Methode und der inversen quadratischen Interpolation.
- **Abbruchbedingung:** Garantiert Konvergenz bis auf die Toleranz $\text{accuracy} = f_{\text{end}} / 10000 + 10^{-5}$ unter Einhaltung von $\varepsilon_{\text{time}} = 10^{-8}$.

3.1.3 Hardware-Mapping & NCO-Synthese

Nach der Bestimmung des Frequenzvektors $f[n]$ werden die Daten in Steuerungsparameter übersetzt. MUFACE v6 verwendet ein dynamisches NCO-Prinzip, das aus einem 16-Bit-Akkumulator (A) und einem Hardware-Teiler (T) besteht:



Die Funktion `berechne_nco_parameter_rampe` löst hierbei eine wesentliche Aufgabe zur Phasen- und Jitter-Reduktion:

1. **Suche des optimalen Teilerverhältnisses:** Der Teiler T wird iterativ durchgesucht, um die Abweichung $\Delta F = |f_{\text{out}} - f_{\text{target}}|$ bei minimalem Phasensprung zu finden.
2. **Akkumulator-Landezonen-Optimierung:**

$$\text{Akkulandezone} = ((\text{Clockimpulse} \cdot A) \bmod A_{\text{max}}) \bmod A$$

Dies stellt sicher, dass der Akkumulator ohne Rest-Phasenfehler überläuft. Dies verhindert Jitter – ein Verhalten, das analog über Boillats Pin-5-Modulation gelöst wurde und nun rein digital abgebildet ist.

3. **Hardware-Schutzschaltung (Anti-Freeze):** Strikte Bereichsgrenzen: $T \in [1, 255]$ und $A \in [0, 65535]$.

4. Quantisierungsfehler & Resonanzvermeidung

4.1 Quantisierungsfehler & Hardware-Schutz (Anti-Freeze)

Beim Anfahren sehr hoher Frequenzen oder beim Abbremsen auf nahe 0 Hz besteht auf 8-Bit- oder 16-Bit-Zielcontroller-Architekturen die Gefahr von Interrupt-Stürmen (bei $T \rightarrow 0$) oder Integer-Überläufen. JavaMotion erzwingt daher strikte Schranken direkt bei der Paketerstellung:

```
if (Teiler[0] > 255) Teiler[0] = 255;
if (Teiler[0] <= 0) Teiler[0] = 1;    // Verhindert Division-by-Zero

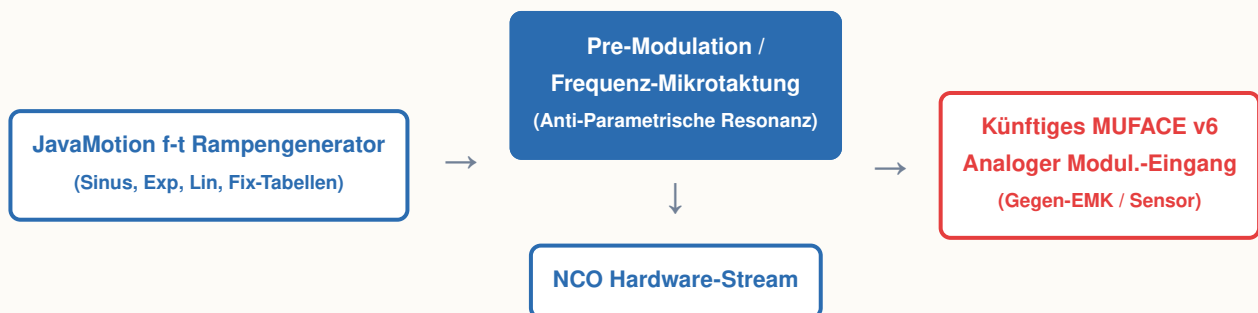
if (Akku[0] > 65535) Akku[0] = 65535;
if (Akku[0] < 0)     Akku[0] = 0;
```

Diese simple, aber wirkungsvolle Schutzschicht verhindert zuverlässig das Einfrieren der Steuerung im laufenden Betrieb.

4.2 Digital-Dither & Vermeidung Parametrischer Resonanzen

In MUFACE v6 übersetzen wir Boillats physikalisches Prinzip gegen parametrische Oszillationen in die digitale Profilberechnung:

- **Unterbrechung parametrischer Kopplung:** Parametrische Resonanzen benötigen eine streng periodische Energiezufuhr. Durch die wählbaren S-Kurven-, Exponential- oder Tabellenprofile wird die Zeitdauer jedes einzelnen Schrittes t_n kontinuierlich verändert, wodurch die parametrische Bedingung $\omega_{Anregung} \approx 2\omega_{Rotor}$ durchbrochen wird. Erst smarte Beschleunigungsrampen ermöglichen das Betreiben der Motoren in ihrem echten physikalischen Effizienz-Maximum.
- **Konzept für Anti-Parametrische Modulation:** Die vor über 26 Jahren entstandene Idee, eine gezielte „Anti-Parametric Modulation“ (in Anlehnung an Boillats Pin-5-Ansatz) in die NCO-Burst- und Rampenberechnungen von JavaMotion einzuweben, bildet das Fundament für künftige Hardware-Erweiterungen.



MUFACE v6 Hardware-Ausblick: Für die kommende MUFACE v6 Hardware-Generation ist ein analoger Modulations-Eingang vorgesehen. Damit kann das Sensorsignal (z. B. Gegen-EMK) analog erfasst und digital verarbeitet werden, um schrittsynchrone Mikromodulationen aufzuprägen und parametrische Rotor-Instabilitäten im Keim zu ersticken.

5. Fazit

Die Evolution der Schrittmotor-Ansteuerung zeigt eine eindrucksvolle Entwicklung: Während Pioniere wie Pierre Boillat mit der SMIK-Methode und NE555-Schaltungen elegante analoge Wege fanden, um Motoren über Pin 5 zu stabilisieren, und Merz (2019) den externen Phasenmodulator für bestehende Steuerungen aufzeigte, verlagert JavaMotion (MUFACE v6) diese Logik in hochpräzise, zeitdiskrete Mathematik. Durch die saubere Trennung von physikalischer Profilberechnung und robuster Hardware-Synthese bietet MUFACE v6 ein praxiserprobtes Framework, das historisches Wissen mit moderner Softwarearchitektur verbindet und Schrittmotoren in ihrem optimalen Wirkungsgradbereich betreibt.

Danksagung / Acknowledgments

Wir danken **Adrian Perrig** und **Marc Fasnacht** (EPFL Lausanne) herzlich für ihre wertvollen wissenschaftlichen Beiträge, die mathematische Beratung bei der Formulierung der iterativen Solver-Verfahren sowie die algorithmische Kernumsetzung (Programmierung) in den frühen Entwicklungsphasen von JavaMotion.

(Redaktionelle Aufbereitung und mathematische Dokumentations-Synthese unter digitaler Assistenz.)

Quellen & Referenzen

- **Boillat, Pierre (1987):** „*The SMIK-method: Stepping Motor Instability Killer*“, Vortrag am Mechatronik Symposium ETH Zürich, 10.–11. September 1987. Johnson Electric International (ehemals Sodeco SAIA AG, Murten).
- **Boillat, Pierre (1987):** „*Stabilizing arrangement for a stepping motor*“, United States Patent US4,683,409. Erteilt an SAIA AG (heute Johnson Electric International).
- **Merz, René (2019):** „*Antiresonanzschaltung für Schrittmotoren: Wirkungsgradsteigerung und Eliminierung von parametrischen Resonanzen*“, Elektor Magazine (Deutsche Ausgabe: [Elektor 87/2019](#); Englische Ausgabe: [Elektor 88/2019](#); Manuskript-Draft 2018).
- **Austin, David (2004):** „*Generate stepper-motor speed profiles in real time*“, EE Times. (Referenz zur zeitdiskreten Taylor-Approximation in ISR-Routinen).
- **JavaMotion Codebase:** Interne Quellcodes (FixRampe.java, LinRampe.java, ExpRampe.java, SinRampe.java, Rampe.java). Historische Entwicklung: Version 1.00 (1999), Version 2.00 (2002), bis hin zu JavaMotion v7.00 (>3.00, 2026) und reinton.ch.